OBJECT ORIENTED PROGRAMMING WITH C++

ASSIGNMENT 5 – CO5

RELIABLE SOFTWARE WITH RUNTIME POLYMORPHISM AND EXCEPTIONS

Student Details	**Fill before submission**
Name	HARIPRASAD R
Register Number	192421179
Course	DSA01 – Object Oriented Programming with C++
Compiler and version	g++ Dev c++
GitHub repository	https://github.com/<username>/DSA01-<RegisterNumber>-Assignment5
Submission date	14-11-2026

Important: This is a model answer and implementation guide. You must compile and execute the programs yourself, record actual timings, capture genuine terminal screenshots, run AddressSanitizer in a supported environment and replace all placeholders with your own details. Do not report invented benchmark values or sanitizer results.

Compilation and Repository Setup

Recommended normal compilation command:

```
g++ -std=c++17 -Wall -Wextra Task.cpp -o Task
```

For performance experiments, compile a separate optimized build:

```
g++ -std=c++17 -O2 -DNDEBUG -Wall -Wextra benchmark.cpp -o benchmark
```

For AddressSanitizer, use a GCC environment that supports the sanitizer runtime:

```
g++ -std=c++17 -g -O1 -Wall -Wextra -fsanitize=address -fno-omit-frame-pointer Task.cpp -o Task_asan
```

A Windows TDM-GCC installation may not provide a working AddressSanitizer runtime. If unavailable, use WSL/Linux or another supported GCC environment and report the actual environment.

Suggested repository structure:

```
DSA01-<RegisterNumber>-Assignment5/\n  README.md\n  PartA/Design-UML-vtables-pseudocode.pdf\n  TaskA1/ ... TaskA5/\n
TaskB1/ ... TaskB4/\n  screenshots/\n  benchmark-results.csv\n  sanitizer-reports/
```

Commit progressively, at least one meaningful commit per task. README must include compiler version, exact compile commands, run commands, and a result note for each task.

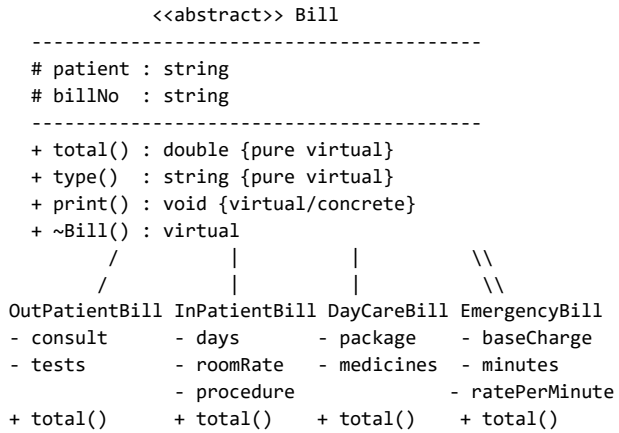
PART A – DESIGN (25 Marks)

TASK A1 – DESIGN THE POLYMORPHIC BILL HIERARCHY (7 Marks)

A1.1 Common Interface

Common data: patient identifier/name, bill number, and billing date may be stored in Bill. The common operations are total() to calculate the amount, type() to identify the bill category, print() to display it, and a virtual destructor. total() and type() are pure virtual because each bill category has a different calculation.

A1.2 Abstract Hierarchy and UML



UML notation: + public, # protected, - private. The base class declares total() and type() as pure virtual (= 0). Each derived class overrides both. print() can be a concrete virtual/non-virtual helper that calls the virtual operations; the destructor must be virtual.

A1.3 Process All Bills – Pseudocode

```
ALGORITHM CalculateDayTotal(bills, count)
BEGIN
    SET dayTotal TO 0
    FOR index FROM 0 TO count - 1
        IF bills[index] IS NOT NULL THEN
            SET dayTotal TO dayTotal + bills[index].total()
        END IF
    END FOR
    RETURN dayTotal
END
```

Alternative: One Structure and a Switch

A single Bill record would contain a type tag and fields for every category: consultation, tests, room days, room rate, procedure, package amount, emergency minutes and emergency rate. total() uses switch(type). For n bills, both approaches take O(n) overall and O(1) per bill calculation when the number of bill types is fixed. The tagged structure wastes space because fields for unrelated categories coexist. Adding a bill type requires changing the enum/type definition, switch, data validation and often construction/reporting code. With polymorphism, a new derived class can implement the interface while the processing loop remains unchanged.

TASK A2 – UNDERSTAND RUNTIME BINDING (5 Marks)

A2.1 Early and Late Binding

Early binding selects a non-virtual function using the expression's static type at compile time. Late binding selects an overridden virtual function at run time using the object's dynamic type. For example, Bill* b = new OutPatientBill(...); b->total() calls OutPatientBill::total() when total() is virtual. A non-virtual call through Bill* selects Bill's version (if one exists).

A2.2 Conceptual Vtable

Class	Conceptual virtual-table entries
Bill	total → pure virtual slot; type → pure virtual slot; destructor → Bill destructor
OutPatientBill	total → OutPatientBill::total; type → OutPatientBill::type; destructor → OutPatientBill destructor
InPatientBill	total → InPatientBill::total; type → InPatientBill::type; destructor → InPatientBill destructor

A vtable is a common compiler implementation technique, not a layout mandated in exact form by the C++ standard. A polymorphic object typically contains a hidden vptr. The call b->total() consults the dynamic object's vptr and dispatches to the derived override.

A2.3 Virtual Destructor

Bill must have a virtual destructor because objects are deleted through Bill*. A virtual destructor ensures that the derived destructor runs first, followed by the base destructor, releasing derived resources correctly. Deleting a derived object through a base pointer when the base destructor is non-virtual has undefined behaviour.

Predict the Output

Write prediction before executing:

```
500 ~OPBill ~Bill
```

show() itself is non-virtual, but its body calls total(). Since total() is virtual, that inner call is dynamically dispatched to OPBill::total(). delete b invokes OPBill::~OPBill() and then Bill::~Bill().

If virtual is removed from ~Bill, deleting through Bill* has undefined behaviour. Common implementations call only the base destructor, so ~OPBill may not appear; this is not a guaranteed output. The shown example owns no extra heap resource, so AddressSanitizer may not report a leak even though the behaviour is undefined. To make the resource leak observable, add an allocated array or unique resource to OPBill and print from its destructor. Record the actual sanitizer result; do not assume a particular diagnostic.

Alternative: dynamic_cast

If total() were non-virtual, code could test each possible type using dynamic_cast<OutPatientBill*>(b), dynamic_cast<InPatientBill*>(b), and so on, then call the matching derived calculation. With k bill types, one bill may require up to k runtime type checks; a virtual call uses one dispatch. Both are O(1) for fixed k, but the cast approach becomes more expensive and repetitive as k grows. A typical polymorphic object uses one hidden vtable pointer (often sizeof(void*)). Exact ABI layout is implementation-dependent.

TASK A3 – DESIGN WITH ABSTRACT CLASSES (5 Marks)

A3.1 Payer Interface

```
#include <algorithm>
#include <string>
#include <unordered_map>
using namespace std;

class Bill {
public:
    virtual double total() const = 0;
    virtual string type() const = 0;
    virtual ~Bill() = default;
};

class Payer {
public:
    virtual double amountCovered(const Bill& bill) const = 0;
    virtual string payerName() const = 0;
    virtual ~Payer() = default;
};

class SelfPay : public Payer {
public:
    double amountCovered(const Bill& bill) const override {
        return bill.total();
    }
    string payerName() const override { return "SelfPay"; }
};

class Insurance : public Payer {
    double coverLimit;
    double coPayPercent; // patient's share, 0..100
public:
    Insurance(double limit, double copay)
        : coverLimit(limit), coPayPercent(copay) {}
    double amountCovered(const Bill& bill) const override {
        const double eligible = min(bill.total(), coverLimit);
        return eligible * (100.0 - coPayPercent) / 100.0;
    }
    string payerName() const override { return "Insurance"; }
};

class GovernmentScheme : public Payer {
    unordered_map<string, double> packageRates;
public:
    explicit GovernmentScheme(unordered_map<string,double> rates)
        : packageRates(move(rates)) {}
    double amountCovered(const Bill& bill) const override {
        auto it = packageRates.find(bill.type());
        if (it == packageRates.end())
            throw runtime_error("unknown government package");
```

```
        return min(bill.total(), it->second);
    }
    string payerName() const override { return "GovernmentScheme"; }
};
```

Policy assumption: coPayPercent is the percentage paid by the patient, so insurance pays the remaining percentage of the eligible amount. The assignment does not prescribe a different interpretation; state this assumption in the report.

A3.2 Extension – Corporate Tie-up

```
class CorporatePayer : public Payer {
    double discountPercent;
    double maximumCover;
public:
    CorporatePayer(double discount, double limit)
        : discountPercent(discount), maximumCover(limit) {}
    double amountCovered(const Bill& bill) const override {
        const double discounted =
            bill.total() * (100.0 - discountPercent) / 100.0;
        return min(discounted, maximumCover);
    }
    string payerName() const override { return "CorporateTieUp"; }
};
```

Add CorporatePayer as a new derived class and store it through Payer*. Existing payer classes and the code that invokes amountCovered() do not need to change.

Alternative: Function Pointers / std::function

A payer policy could be represented as std::function<double(const Bill&)> plus a label and captured configuration. For example, a lambda captures coverLimit and coPayPercent. Each policy call is O(1) for a simple formula, just like a virtual call. std::function may require a type-erasure wrapper and sometimes a heap allocation; exact storage is implementation-dependent. An abstract Payer object naturally groups policy data and behaviour in one named type, while a function object stores data in a closure and behaviour in its callable target.

TASK A4 – DESIGN THE EXCEPTION STRATEGY (4 Marks)

A4.1 Error List and Exception Hierarchy

Error condition	Suggested exception / handling
Negative consultation, room, medicine or package charge	InvalidCharge : BillingError
Claim above policy cover	ClaimError : BillingError; reject or calculate covered portion according to policy
Unknown government package code	UnknownPackage : BillingError
Missing patient record	PatientNotFound : BillingError
Text supplied where a number is expected	std::invalid_argument from std::stod
Impossible / failed allocation	std::bad_alloc
Invalid days, minutes or percentage	std::invalid_argument or InvalidCharge

```
class BillingError : public std::runtime_error {
public:
    using std::runtime_error::runtime_error;
};
class InvalidCharge : public BillingError {
public: using BillingError::BillingError;
};
class ClaimError : public BillingError {
public: using BillingError::BillingError;
};
class UnknownPackage : public BillingError {
public: using BillingError::BillingError;
};
```

```
class PatientNotFound : public BillingError {
public: using BillingError::BillingError;
};
```

A4.2 Throwing and noexcept

May throw: constructors that allocate or validate input; total()/amountCovered() if they validate or look up a missing package; parsing functions; claim approval; file/database operations; vector growth; and functions that explicitly report billing errors. Mark a function noexcept only when it truly cannot allow an exception to escape. Simple accessors and destructors should normally be noexcept. Destructors must not throw during stack unwinding; a second exception escaping while another exception is active can call std::terminate().

Do not mark a function noexcept merely to silence warnings. If an operation can allocate or call throwing code, either allow it to throw or catch and handle the failure internally.

TASK A5 – DESIGN RECOVERY AND RETHROWING (4 Marks)

A5.1 Where to Catch

```
ALGORITHM ProcessClaim(claim)
BEGIN
    TRY
        Validate patient and bill
        Calculate bill total
        Calculate payer coverage
        Store approved/rejected claim
    CATCH ClaimError e
        Log claim identifier and reason
        RETHROW the same exception
    CATCH BillingError e
        Log billing context
        RETHROW the same exception
    END TRY
END

ALGORITHM MainRequestHandler(request)
BEGIN
    TRY
        CALL ProcessClaim(request)
        Display successful result
    CATCH BillingError e
        Display safe billing error to user
    CATCH std::exception e
        Log unexpected technical error
        Display generic failure message
    CATCH (...)
        Log unknown failure
        Display generic failure message
    END TRY
END
```

Catch close to the layer that has useful context for logging and cleanup; rethrow with throw; when a higher layer must decide the user-facing response. Catch at the application boundary so the process can report failure safely.

A5.2 Stack Unwinding and RAII

When an exception leaves a scope, C++ destroys fully constructed automatic objects in reverse construction order. This is stack unwinding. RAII ties resource ownership to object lifetime: std::vector, std::string, std::unique_ptr, std::fstream and lock guards release their resources in destructors even when an exception interrupts normal control flow.

Predict the Output

```
~f ~main caught 42
```

Guard g is destroyed while the exception leaves f(). Then the exception continues to main and Guard m is destroyed during unwinding. The statement printing “not printed” is skipped because throw transfers control to a matching handler. The int handler prints caught 42.

Alternative: Manual delete in Catch Blocks

Manual cleanup requires every error path to release every resource. Runtime cost is broadly similar for the actual release operation, but code is larger and fragile. Each newly added throw point creates another place where a missing delete, close or unlock can leak a resource. RAII centralizes cleanup in destructors and is safer.

PART B – IMPLEMENTATION & EXPERIMENTAL INVESTIGATION (25 Marks)

TASK B1 – IMPLEMENT AND VALIDATE POLYMORPHIC BILLING (7 Marks)

B1.1 Complete the Code – TODO 1 to TODO 3

```
#include <iostream>
#include <string>

class Bill {
protected:
    std::string patient;
public:
    explicit Bill(const std::string& p) : patient(p) {}
    virtual ~Bill() = default;

    virtual double total() const = 0;
    virtual std::string type() const { return "Bill"; }

    void print() const {
        std::cout << type() << ' ' << patient << ": " << total() << '\\n';
    }
};

class OutPatient : public Bill {
    double consultation, tests;
public:
    OutPatient(const std::string& p, double c, double t)
        : Bill(p), consultation(c), tests(t) {}
    double total() const override { return consultation + tests; }
    std::string type() const override { return "OP"; }
};

class InPatient : public Bill {
    int days;
    double perDay, surgery;
public:
    InPatient(const std::string& p, int d, double rate, double s)
        : Bill(p), days(d), perDay(rate), surgery(s) {}
    double total() const override { return days * perDay + surgery; }
    std::string type() const override { return "IP"; }
};

int main() {
    Bill* bills[] = {
        new OutPatient("Ravi", 500, 1200),
        new InPatient("Meena", 3, 2500, 40000)
    };
    double sum = 0;
    for (Bill* b : bills) {
```

```
        b->print();
        sum += b->total();
    }
    std::cout << "Total " << sum << '\\n';
    for (Bill* b : bills) delete b;
}
```

Expected output:

```
OP Ravi: 1700\nIP Meena: 47500\nTotal 49200
```

B1.2 Compare with Design

The implementation follows the design: Bill is abstract, total() is pure virtual, derived classes override total() and type(), and the base destructor is virtual. print() is shared and calls the virtual functions.

B1.3 Remove virtual from total()

If total() remains pure virtual but the virtual keyword is removed, the declaration is no longer a virtual function and cannot be a pure virtual function. The compiler rejects the declaration (a pure specifier is only valid for a virtual function). If instead a non-virtual base total() implementation is provided, calls through Bill* bind to that base implementation, not the derived calculation.

B1.4 Remove Virtual Destructor

Deleting derived objects through Bill* then has undefined behaviour. A derived destructor may be skipped, causing owned resources to leak. AddressSanitizer may report leaks if the derived class owns heap memory; the minimal example above has no derived allocation, so the absence of a leak report does not prove correctness.

Alternative: Templates

Compile-time polymorphism can use template functions for each concrete bill type, such as template<class T> sumBills(const vector<T>&). It avoids virtual dispatch, but requires separate containers or a common variant for heterogeneous bills. A single list of different unrelated derived types cannot be represented directly as vector<T> without type erasure or a tagged union. Per bill processing remains O(1); iterating n bills remains O(n).

TASK B2 – ENGINEER ERROR CONDITIONS (5 Marks)

Predict the Output

```
ok done caught negative [log too large] caught too large
```

For 100, validate prints “ok” and process returns, so main prints “done”. For -5, validate throws invalid_argument; process catches only out_of_range, so the exception goes directly to main and is caught by const std::exception. For 2e6, validate throws out_of_range; process catches it, prints the log message, and rethrows with throw;. Main then catches it. Only the out_of_range exception is logged because process has no invalid_argument handler.

Complete the Code – TODO 1 and TODO 2

```
#include <iostream>
#include <stdexcept>

class ClaimError : public std::runtime_error {
public:
    using std::runtime_error::runtime_error;
};

double approve(double billed, double cover) {
    if (billed < 0 || cover < 0)
        throw std::invalid_argument("negative amount");
    if (billed > cover)
        throw ClaimError("exceeds cover");
```

```
    return billed;
}

double settle(double billed, double cover) {
    try {
        return approve(billed, cover);
    } catch (const ClaimError& e) {
        std::cout << "audit: " << e.what() << '\\n';
        throw;
    }
}

int main() {
    double cases[][2] = {{40000, 50000}, {60000, 50000}, {-1, 50000}};
    for (auto& c : cases) {
        try {
            double paid = settle(c[0], c[1]);
            std::cout << "approved " << paid << '\\n';
        } catch (const ClaimError& e) {
            std::cout << "claim rejected: " << e.what() << '\\n';
        } catch (const std::exception& e) {
            std::cout << "error: " << e.what() << '\\n';
        }
    }
}
```

Expected output:

```
approved 40000
audit: exceeds cover
claim rejected: exceeds cover
error: negative amount
```

Replacing throw; with throw e; throws a new exception object whose static type is the caught expression type ClaimError (or may slice further if caught by a base reference). In this exact code the caught type is ClaimError, so no slicing occurs, but the original exception object identity and dynamic exception type are not preserved in general. throw; rethrows the currently handled exception unchanged and is preferred for rethrowing.

B2 Error Test Plan

Test input	Expected behaviour	What it demonstrates
Negative charge	InvalidCharge / invalid_argument caught and explained	Validation before calculation
Claim > cover	ClaimError logged and rethrown to boundary	Layered recovery
Unknown package	UnknownPackage caught at request boundary	Explicit lookup failure
Text instead of number	std::stod throws invalid_argument	Input parsing protection
Very large allocation	std::bad_alloc if allocation really fails	Allocation failure handling
catch(...) at top	Unknown exception is reported; RAII destructors run	Last-resort boundary and cleanup

Forcing allocation failure is environment-dependent. Requesting an impossibly large array may throw std::bad_array_new_length or std::bad_alloc, or the operating system may terminate the process under memory pressure. Use a controlled test and catch std::bad_alloc; do not exhaust the machine memory.

Alternative: std::optional

approveOptional() can return optional<double>; nullopt means rejected. The caller checks has_value() and obtains the amount. Both approaches have O(1) work per claim and O(n) for one million claims. Return codes/optional avoid exception-unwinding cost for expected invalid data; exceptions are generally efficient on the normal path but can be much more expensive when thrown. At 0% invalid, exceptions usually have low overhead; at 1% and especially 50%, exception handling can become slower. optional stores a value plus a

presence tag (with padding); an exception uses runtime-managed exception storage only on the error path. Benchmark the actual compiler rather than inventing times.

TASK B3 – VIRTUAL FUNCTIONS vs switch vs EXCEPTIONS: THE COST (8 Marks)

B3.1 Correctness on 10 Bills

Use the same input data in both implementations and compare the accumulated total. Example test set: five outpatient bills with totals 500, 1700, 2200, 900 and 1200; five inpatient bills with totals 5000, 8000, 12000, 6500 and 9500. Expected total = 57,500. The two implementations must print the same total.

B3.2 Benchmark Virtual vs Switch

Use a deterministic input generator, run each version several times, and report the median. Keep construction outside the timed region if measuring only dispatch/calculation. Use -O2, warm up once, and prevent the compiler from removing the result by printing or accumulating a checksum.

```
// Benchmark outline (implement both loops using identical generated values)
for (long long n : {1000LL, 1000000LL, 10000000LL}) {
    auto data = makeIdenticalBills(n);
    auto start = steady_clock::now();
    volatile double checksum = 0;
    for (const auto& b : polymorphicBills) checksum += b->total();
    auto elapsed = steady_clock::now() - start;
    report("virtual", n, elapsed, checksum);

    start = steady_clock::now();
    checksum = 0;
    for (const auto& b : taggedBills) checksum += switchTotal(b);
    elapsed = steady_clock::now() - start;
    report("switch", n, elapsed, checksum);
}
```

Record a table with N, virtual median milliseconds, switch median milliseconds, checksum and hardware/compiler details. Repeat at least five times. A virtual version may incur indirect-call and pointer/locality costs; a switch may benefit from contiguous data and compiler optimization. Results depend on layout, branch predictability and compiler.

B3.3 Exception Cost

Compare a return-code validator and an exception validator over 1,000,000 pre-generated claims with invalid ratios 0%, 1% and 50%. Do not generate random data inside the timed loop. For return codes, return a status and output value. For exceptions, throw only for invalid claims and catch around each claim (or use a controlled batch boundary). Both process O(n) claims; exception cost is concentrated in invalid cases.

Invalid ratio	Number of invalid claims (N=1,000,000)	Expected qualitative result
0%	0	Exception path is never entered; often close to return-code path
1%	10,000	Exception design pays for 10,000 throws/catches; likely slower
50%	500,000	Frequent throws can make exception design substantially slower

B3.4 Engineering Trade-Off

Use virtual functions when a stable common interface must support heterogeneous objects and extension by new derived types. Use a switch/tagged structure when the set of types is small and fixed and compact data locality matters. Use exceptions for exceptional failures such as missing records, I/O failures and allocation errors; use return values/optional for frequent expected outcomes such as a routine claim rejection.

Alternative: std::variant and std::visit

```
using AnyBill = std::variant<OutPatientBill, InPatientBill, DayCareBill>;
double totalOf(const AnyBill& b) {
    return std::visit([](const auto& bill) { return bill.total(); }, b);
}
```

variant stores enough inline space for its largest alternative plus a discriminator and alignment padding. std::visit performs type-based dispatch and can be optimized by the compiler; measure it against virtual and switch implementations at 10,000,000 elements. Unlike a base-pointer list, variant stores values directly and avoids one heap allocation per bill when stored in vector<variant<...>>. However, the set of alternatives is closed: adding a type changes the variant declaration and visitors.

TASK B4 – FINAL ENGINEERING DECISION (5 Marks)

B4.1 Add a New Bill and Payer

```
class DayCareBill : public Bill {
    double packagePrice;
public:
    DayCareBill(string patient, double price)
        : Bill(move(patient)), packagePrice(price) {}
    double total() const override {
        if (packagePrice < 0) throw InvalidCharge("negative day-care package");
        return packagePrice;
    }
    string type() const override { return "DayCare"; }
};

class CorporatePayer : public Payer {
    double coverLimit;
public:
    explicit CorporatePayer(double limit) : coverLimit(limit) {}
    double amountCovered(const Bill& bill) const override {
        return min(bill.total(), coverLimit);
    }
    string payerName() const override { return "Corporate"; }
};
```

Add a DayCareBill object and CorporatePayer object to the same base-pointer processing code. The existing OutPatientBill, InPatientBill and payer classes should remain unchanged. Add tests that rerun all earlier bill and payer combinations.

B4.2 Resource Safety

Run the complete tests with AddressSanitizer and include the actual report. A clean report means the tested run completed without sanitizer-detected memory errors or leaks; it does not prove that every possible path is safe. Include tests that throw and catch exceptions while std::vector, std::string and std::unique_ptr objects are alive, to demonstrate RAII cleanup.

B4.3 Production Decision

A polymorphic bill and payer design with exceptions is a suitable starting architecture for a multi-hospital billing service because bill and payer categories are open to extension, common processing is uniform, and RAII plus virtual destructors support safe resource ownership. Before production, add persistent transaction storage, audit logging, authentication/authorization, currency and rounding rules, idempotency, concurrency control, policy versioning, and comprehensive tests. For 200,000 bills/day, O(n) processing is reasonable in principle, but the decision must be supported by the measured benchmark and database/network performance; do not infer production capacity from a microbenchmark alone.

Alternative: Configuration / Plug-in Rules

Configuration-driven rules store bill and payer rates in validated configuration files or a database. A rule interpreter performs lookups and evaluates formulas; a simple rule can be O(1) per bill after indexing, while complex interpreted rules may cost more than a virtual call. Rule storage is O(r) for r rules. Configuration changes can add rates without recompilation, but arbitrary new behaviour may require a new rule

operator or plug-in code. Validate and version rules carefully because a bad configuration can affect every hospital. C++ derived classes are safer for complex new algorithms but require software deployment; configuration is convenient for rate-only changes.

Final Submission Checklist

- Complete the cover page and add the real GitHub repository link.
- Create DSA01-<RegisterNumber>-Assignment5 and share it with the faculty.
- Include Part A UML, vtable concept diagram, exception hierarchy, pseudocode and reasoning.
- Save every completed TODO program and all other source files in task folders.
- Write predicted outputs before running the programs.
- Compile with g++ -std=c++17 -Wall -Wextra and fix warnings.
- Run AddressSanitizer in a supported environment; include genuine reports for both defective and corrected cases.
- Benchmark with -O2 for all required sizes and invalid ratios; include median timings and environment details.
- Add one new bill type and one new payer without modifying existing classes; rerun regression tests.
- Commit progressively and include README.md, input data, screenshots and benchmark-results.csv.
- Prepare to explain vtables, late binding, virtual destructors, stack unwinding, rethrowing and RAII in the viva.